PES UNIVERSITY

100 Feet Ring Road, BSK 3rd Stage, Bengaluru – 560085

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(AI & ML)
B. TECH. CSE – 5th SEMESTER
UE23CS351A
DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT ON

SMART RESEARCH & PEER REVIEW PORTAL

A Hybrid SQL and NoSQL-Based Academic Research Management System

Submitted by

Student Name	SRN
Prashant Devendra Naik	PES1UG24AM199
Pradyun P Urs	PES1UG24AM195

Aug – Dec 2026 | Academic Year 2026–2027

ABSTRACT

The Smart Research & Peer Review Portal is an academic research management application intended to organize research manuscript submission, reviewer allocation, manuscript evaluation, and feedback delivery. The application provides capabilities for students, reviewers, and administrative users.

The confirmed relational database is implemented using PostgreSQL 18. It stores structured information about departments, users, research topics, manuscripts, reviewer profiles, assignments, reviews, and audit logs. The Next.js application communicates with PostgreSQL through the Node.js **pg** connection pool. SQL queries retrieve user information, connect related records, and display review results.

The review workflow connects a student's manuscript to reviewer assignments and submitted reviews. The student review page/API is designed to retrieve review details using the authenticated student's user ID. The

database demonstrates primary keys, foreign keys, constraints, joins, aggregate calculations, a view, a stored function, a trigger, and audit logging.

MongoDB discussion data is shown in the supplied MongoDB Compass evidence; application-level CRUD integration should be verified separately. pgvector semantic search is not claimed as operational because its extension installation was previously blocked by a Windows application-control policy.

TABLE OF CONTENTS

Sl. No.	Topic	Page
1	Introduction and Problem Statement	3
2	User Requirements Specification – Technology Stack	4
3	E-R Model	5
4	Relational Model	6
5	SQL DDL Statements	7
6	SQL DML Statements	8
7	NoSQL / Vector Database Commands	9
8	Results – Resulting Tables' Screenshots	10
9	Next.js Project Tree Structure	11
10	Application Code and Demo Screenshots	12
11	Evaluation Rubrics – Completion Status	13

Page numbers are indicative; regenerate the table of contents if the report is edited further.

1. INTRODUCTION

Research paper evaluation involves multiple participants and related activities. A student prepares and submits a manuscript, an administrator allocates it to a suitable reviewer, and the reviewer evaluates the work and submits feedback. When these activities are managed separately, it becomes difficult to track manuscript status, assignments, review history, and feedback.

The Smart Research & Peer Review Portal brings these activities into one web application. The database maintains relationships among users, manuscripts, reviewers, assignments, and reviews. The application uses a browser-based interface and backend API routes to perform database operations.

1.1 Problem Statement

The project addresses the need for a centralized system to manage academic research submissions and peer-review activities. The system must maintain manuscript information, identify the student who submitted each manuscript, record reviewer assignments, store evaluation results, and allow the correct student to view feedback. It must preserve data integrity through database keys and constraints and maintain an audit record of manuscript status changes.

1.2 Objectives

- Provide a central portal for academic manuscript submission and review.
- Maintain student, reviewer, administrator, department, and research-topic information.
- Connect manuscripts with assigned reviewers.
- Store technical scores, originality scores, comments, and recommendations.

- Allow students to view evaluations associated with their own manuscripts.
- Use relational database constraints to maintain data integrity.
- Demonstrate SQL joins, functions, triggers, views, and audit logging.

1.3 Scope

The confirmed relational scope includes the PostgreSQL schema and its nine physical tables. The Next.js application includes authentication-related code and a student review page/API route. MongoDB-backed discussions and pgvector semantic search should be described as completed application features only after their integration is verified in the running application.

1.4 Proposed Workflow

```
Student signs in and submits a manuscript
|                 v Manuscript stored in
PostgreSQL                  |                 v
Administrator assigns a reviewer
|                 v Reviewer evaluates
manuscript                  |                 v
Review scores, comments and recommendation saved
|                 v Student dashboard retrieves
own review information                  |
v
Manuscript status update recorded in audit_logs
```

2. USER REQUIREMENTS SPECIFICATION

2.1 Functional Requirements

- Users can register and sign in to the portal.
- Students can submit and manage manuscript information.
- Reviewer profiles contain expertise information.
- Assignments connect manuscripts with reviewers and due dates.
- Reviewers can record technical and originality scores, comments, and recommendations.
- Students can view review information associated with their own manuscripts.
- The database records manuscript status changes in an audit table.

2.2 Non-Functional Requirements

- Security: protected routes must validate the authenticated session and role.
- Integrity: primary keys, foreign keys, unique constraints, and NOT NULL constraints protect relational data.
- Maintainability: frontend pages, API routes, and database helpers are separated.
- Usability: role-oriented pages provide a clear research workflow.
- Reliability: database operations should handle errors and preserve valid records.

2.3 Technology Stack and Justification

Technology	Role	Justification
Next.js App Router	Frontend pages and backend API routes	Provides a unified web application and server-side route handlers.
React	Interactive user interface	Supports reusable interface components and stateful dashboards.

TypeScript	Application logic	Adds type checking to application code.
Tailwind CSS	Styling	Supports consistent, responsive interface styling.
Node.js	Server runtime	Runs the Next.js server-side application.
PostgreSQL 18	Relational database	Provides relational tables, constraints, joins, views, functions, and triggers.
pg	Database connectivity	Provides connection pooling and parameterized SQL execution.
JOSE	Session token handling	Used by the authentication helper to create and verify signed session tokens.
MongoDB	Document-oriented data	Suitable for flexible discussion/message documents whose fields may evolve independently. Integration status must be verified.

2.4 NoSQL Choice Justification

MongoDB is a document-oriented database. Review discussion or message records can contain flexible fields such as manuscript identifier, student and reviewer names, discussion text, discussion type, timestamp, and status. A document model can represent each discussion as a single record without requiring a fixed relational column set for every possible discussion attribute. The supplied MongoDB Compass evidence shows a database named **smart_research_portal** and a collection named **review_discussions**. The screenshot demonstrates stored documents; it does not alone prove that every application workflow performs MongoDB CRUD operations.

3. E-R MODEL

The ER model represents the major entities in the academic research workflow and the relationships among them. The database contains nine physical PostgreSQL tables.

3.1 List of Entities

Entity	Type	Purpose
DEPARTMENTS	Strong	Department names and codes.
USERS	Strong	Account identity, email, password hash, role, and department.
RESEARCH_TOPICS	Strong	Research topic names and descriptions.
MANUSCRIPTS	Strong	Research paper title, abstract, keywords, student owner, status, and timestamps.
REVIEWERS	Strong	Reviewer profile linked to a user account; expertise and qualification.
MANUSCRIPT_TOPICS	Associative / weak-style	Resolves the many-to-many relationship between manuscripts and research topics.
ASSIGNMENTS	Strong / associative	Connects a manuscript to a reviewer and stores assignment details.
REVIEWS	Strong dependent record	Stores the evaluation associated with an assignment.
AUDIT_LOGS	Dependent log entity	Stores recorded actions and manuscript status-change details.

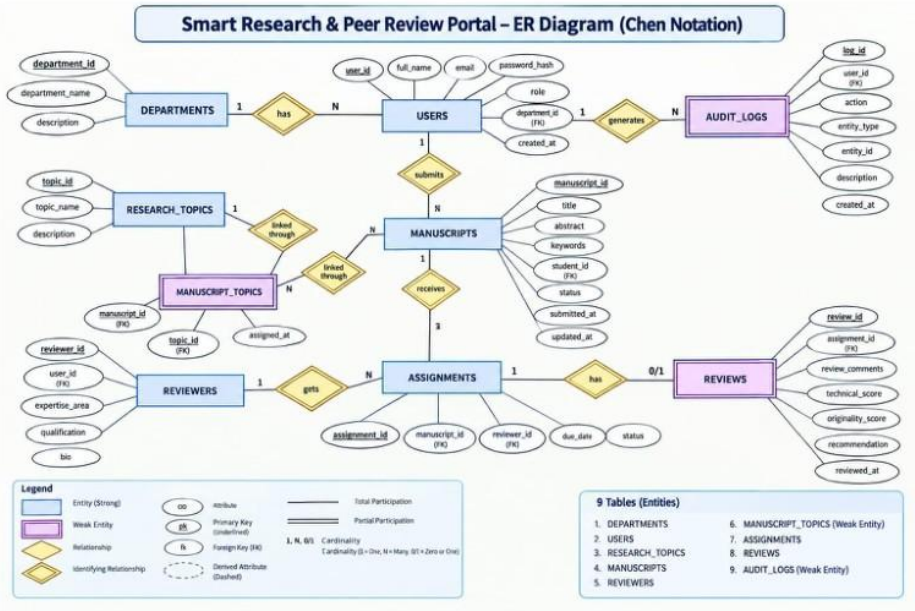
3.2 Non-Simple / Composite Attributes

The supplied relational schema primarily uses atomic columns. At the conceptual level, a user profile can be viewed as a collection of name, email, role, and department details. A manuscript record groups title, abstract, keywords, status, and timestamps. These are separate attributes in the relational implementation rather than one composite database column.

3.3 ER Diagram Snapshot

The diagram below is reproduced from the supplied project report. It shows the nine entities, primary-key/foreign-key attributes, and relationships such as department–user, user–manuscript, manuscript–assignment, reviewer–assignment, and assignment–review.

SMART RESEARCH & PEER REVIEW PORTAL



10. ENTITY AND RELATIONSHIP DESIGN

The database is organized around users, manuscripts, reviewers, assignments, and reviews. A user can submit multiple manuscripts. Each manuscript belongs to one student account through manuscripts.student_id. A manuscript can have reviewer assignments. Each assignment connects one manuscript to one reviewer. A submitted review is connected to its assignment through reviews.assignment_id.

The manuscript_topics table resolves the many-to-many relationship between manuscripts and research_topics. The composite primary key prevents the same topic association from being entered more than once for the same manuscript.

The audit_logs table records system actions. The known manuscript status trigger inserts a log entry when a manuscript status is updated.

11. POSTGRESQL – SQL DATABASE COMMANDS

11.1 Connect to the database

```
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -d smartportal_db
```

11.2 List tables and inspect schema

```
\dt
\d users
\d manuscripts
```

PES University | DBMS Project Report | Page 8

Figure 3.1 – ER diagram evidence from the existing project report (page 8).

4. RELATIONAL MODEL

4.1 Relations

The relational model maps the ER entities and associative relationships into PostgreSQL tables. The relation **manuscript_topics** resolves the many-to-many relationship between manuscripts and research topics. **assignments** represents the connection between a manuscript and a reviewer. These are associative relations in the relational design.

4.2 Relational Schema Summary

Relation	Primary Key	Important Foreign Keys
departments	department_id	—
users	user_id	department_id → departments
research_topics	topic_id	—
manuscripts	manuscript_id	student_id → users
reviewers	reviewer_id	user_id → users
manuscript_topics	(manuscript_id, topic_id)	manuscript_id → manuscripts; topic_id → research_topics
assignments	assignment_id	manuscript_id → manuscripts; reviewer_id → reviewers
reviews	review_id	assignment_id → assignments
audit_logs	log_id	user_id → users (nullable)

4.3 Important Constraints

- users.email is unique.
- departments.department_name and department_code are unique.
- research_topics.topic_name is unique.
- reviewers.user_id is unique.
- assignments has a unique constraint on (manuscript_id, reviewer_id).
- reviews.assignment_id is unique.
- manuscript_topics uses a composite primary key (manuscript_id, topic_id).

4.4 Schema Diagram

The supplied ER diagram is used as the schema relationship reference. It shows the foreign-key paths among the nine physical relations. For a final submission, a separate relational schema diagram generated from the actual PostgreSQL catalog may be added if required by the faculty.

5. SQL DDL STATEMENTS

Data Definition Language (DDL) statements define the database structure. The following representative statement creates the departments relation. The actual project database contains nine physical tables; inspect the live schema before treating this excerpt as a complete schema export.

```
CREATE TABLE departments (     department_id INTEGER GENERATED
ALWAYS AS IDENTITY PRIMARY KEY,     department_name VARCHAR(100)
NOT NULL UNIQUE,     department_code VARCHAR(20) NOT NULL UNIQUE,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

5.1 Database and Schema Verification

```
-- Connect to the project database
\c smartportal_db

-- List all physical tables
\dt

-- Inspect important relations
\d users
\d manuscripts
\d assignments
\d reviews
```

5.2 Primary Key and Foreign Key Examples

```
-- Example relationship:
FOREIGN KEY (student_id)
REFERENCES users(user_id);

-- Composite primary key in manuscript_topics:
PRIMARY KEY (manuscript_id, topic_id);
```

The identity columns generated ALWAYS should normally be omitted from INSERT statements unless an explicit override is intentionally used. The supplied report documents the existing schema and key relationships; a complete reproducible DDL export should be generated from the current database for deployment.

6. SQL DML STATEMENTS

Data Manipulation Language (DML) statements insert, update, and delete records. Data Query Language (DQL) retrieves records.

6.1 INSERT

```
INSERT INTO research_topics (topic_name, description)
VALUES ('Artificial Intelligence', 'Research related to AI');
```

6.2 UPDATE

```
UPDATE manuscripts
SET status = 'UNDER_REVIEW'
WHERE manuscript_id = 1;
```

The project's manuscript status trigger is associated with UPDATE operations on the manuscripts table. A real update should be performed only when intentionally testing the workflow.

6.3 SELECT

```
SELECT *
FROM manuscripts
ORDER BY manuscript_id;
```

6.4 JOIN Query

```
SELECT
    m.manuscript_id,
    m.title AS manuscript_title,
    u.full_name AS student_name,
    u.email AS student_email,
    m.status FROM
manuscripts m
INNER JOIN users u
    ON m.student_id = u.user_id
ORDER BY m.manuscript_id;
```

6.5 GROUP BY and Aggregate Functions

```
SELECT status, COUNT(*) AS total_manuscripts
FROM manuscripts
GROUP BY status;

SELECT
    AVG(technical_score) AS average_technical_score,
    AVG(originality_score) AS average_originality_score,
    COUNT(*) AS total_reviews
FROM reviews;
```

6.6 View Query

```
SELECT *
FROM manuscript_review_summary;
```

6.7 Transaction Example

```
BEGIN;

UPDATE manuscripts
SET status = 'UNDER_REVIEW'
```

```
WHERE manuscript_id = 1;

COMMIT;

-- Use ROLLBACK instead of COMMIT to cancel
-- uncommitted changes during a test.
```

The transaction block demonstrates PostgreSQL transaction syntax. Explicit use of this exact block in the application workflow has not been independently verified.

7. NOSQL / VECTOR DATABASE COMMANDS

7.1 MongoDB

The supplied MongoDB Compass screenshot shows database **smart_research_portal** and collection **review_discussions**. The visible documents include manuscript_id, student_name, discussion, reviewer_name, discussion_type, created_at, and status.

```
// MongoDB shell
examples show dbs use
smart_research_portal
show collections

db.review_discussions.find()
db.review_discussions.findOne()

// Example filter
db.review_discussions.find(
{     manuscript_id: 1 })
```

These commands are examples for inspecting the collection shown in Compass. Application-level create, read, update, and delete integration must be validated separately.

7.2 Vector Database / pgvector Status

pgvector was considered for semantic similarity search. During the recorded Windows setup, PostgreSQL blocked vector.dll under the system's application-control policy. Therefore, pgvector semantic search is **pending** and is not presented as an operational feature.

```
-- Only run after pgvector is installed and allowed
-- by the operating-system application-control
policy:
CREATE EXTENSION vector;

-- Verify extension availability:
SELECT name, default_version, installed_version
FROM pg_available_extensions
WHERE name = 'vector';
```

Do not report vector search as completed unless the extension can be created successfully and a real vector similarity query has been executed.

8. RESULTS – RESULTING TABLES' SCREENSHOTS

The following evidence is taken from the supplied project report. It demonstrates the actual PostgreSQL table listing, ER design, JOIN output, trigger/function and audit records, and MongoDB Compass collection.

SMART RESEARCH & PEER REVIEW PORTAL

	correctly.
Audit trigger	Updating manuscript status creates an audit_logs record.

Record the actual outcome of each test before submission. Do not mark a test as passed without running it.

19. SCREENSHOTS AND EVIDENCE

Use only the following essential screenshots to keep the report focused. Crop each screenshot so that the relevant query, table, or application area is readable. Hide passwords, tokens, private email addresses, and other sensitive details.

Screenshot 1 – PostgreSQL table list

```
smartportal_db=# \dt
                List of tables
 Schema |       Name        | Type  |  Owner
--------+-------------------+-------+----------
 public | assignments       | table | postgres
 public | audit_logs        | table | postgres
 public | departments       | table | postgres
 public | manuscript_topics | table | postgres
 public | manuscripts       | table | postgres
 public | research_topics   | table | postgres
 public | reviewers         | table | postgres
 public | reviews           | table | postgres
 public | users             | table | postgres
(9 rows)
```

Screenshot 2 – ER diagram

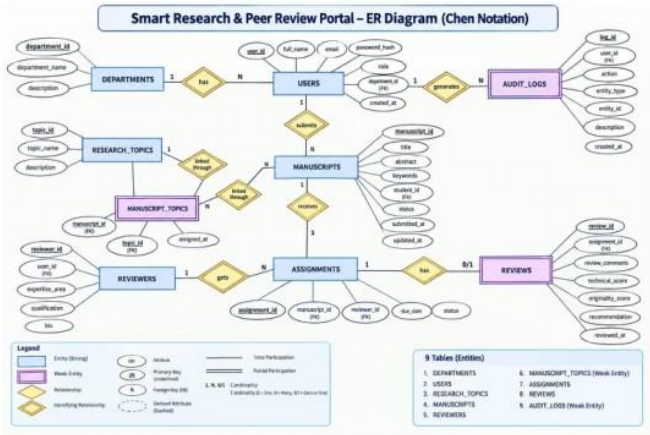


Figure 8.1 – PostgreSQL table listing and ER diagram evidence.

SMART RESEARCH & PEER REVIEW PORTAL

Screenshot 3 – SQL JOIN output

```
smartportal_db=# SELECT
smartportal_db-#     m.manuscript_id,
smartportal_db-#     m.title AS manuscript_title,
smartportal_db-#     u.full_name AS student_name,
smartportal_db-#     u.email AS student_email,
smartportal_db-#     m.status
smartportal_db-# FROM manuscripts m
smartportal_db-# INNER JOIN users u
smartportal_db-# ON m.student_id = u.user_id
smartportal_db-# ORDER BY m.manuscript_id;
 manuscript_id |             manuscript_title             | student_name |    student_email     |    status
---------------+------------------------------------------+--------------+----------------------+--------------
             1 | AI Based Crop Disease Detection          | Rahul Sharma | rahul@example.com    | UNDER_REVIEW
             2 | Smart Traffic Management System          | Ananya Rao   | ananya@example.com   | UNDER_REVIEW
             3 | IoT Based Smart Campus Monitoring System | Rahul Sharma | rahul@example.com    | SUBMITTED
```

Screenshot 4 – Trigger / audit evidence

```
smartportal_db-#     SELECT
smartportal_db-#     trigger_name,
smartportal_db-#     event_object_table,
smartportal_db-#     event_manipulation,
smartportal_db-#     action_timing
smartportal_db-# FROM information_schema.triggers
smartportal_db-# WHERE event_object_table = 'manuscripts';
        trigger_name         | event_object_table | event_manipulation | action_timing
-----------------------------+--------------------+--------------------+---------------
 manuscript_status_audit_trigger | manuscripts        | UPDATE             | AFTER
(1 row)


smartportal_db=# SELECT pg_get_functiondef(
smartportal_db(#     'public.log_manuscript_status_change()'::regprocedure
smartportal_db(# );
                pg_get_functiondef
-------------------------------------------------
 CREATE OR REPLACE FUNCTION public.log_manuscript_status_change()+
  RETURNS trigger                                               +
  LANGUAGE plpgsql                                              +
 AS $function$                                                  +
 BEGIN                                                          +
     INSERT INTO audit_logs (                                   +
         user_id,                                               +
         action,                                                +
         entity_type,                                           +
         entity_id,                                             +
         description                                            +
     )                                                          +
     VALUES (                                                   +
         NULL,                                                  +
         'STATUS_CHANGED',                                      +
         'MANUSCRIPT',                                          +
         NEW.manuscript_id,                                     +
         'Manuscript status changed from '                      +
         || OLD.status || ' to ' || NEW.status                  +
     );                                                         +
                                                                +
     RETURN NEW;                                                +
 END;                                                           +
 $function$                                                     +

(1 row)


smartportal_db=# SELECT *
smartportal_db-# FROM audit_logs
smartportal_db-# ORDER BY log_id;
 log_id | user_id |    action    | entity_type | entity_id |                   description                    |         created_at
--------+---------+--------------+-------------+-----------+--------------------------------------------------+----------------------------
      1 |         | STATUS_CHANGED | MANUSCRIPT  |         1 | Manuscript status changed from SUBMITTED to UNDER_REVIEW | 2026-10-30 19:11:45.174675
      2 |         | STATUS_CHANGED | MANUSCRIPT  |         3 | Manuscript status changed from SUBMITTED to UNDER_REVIEW | 2026-10-30 19:12:32.738249
(2 rows)

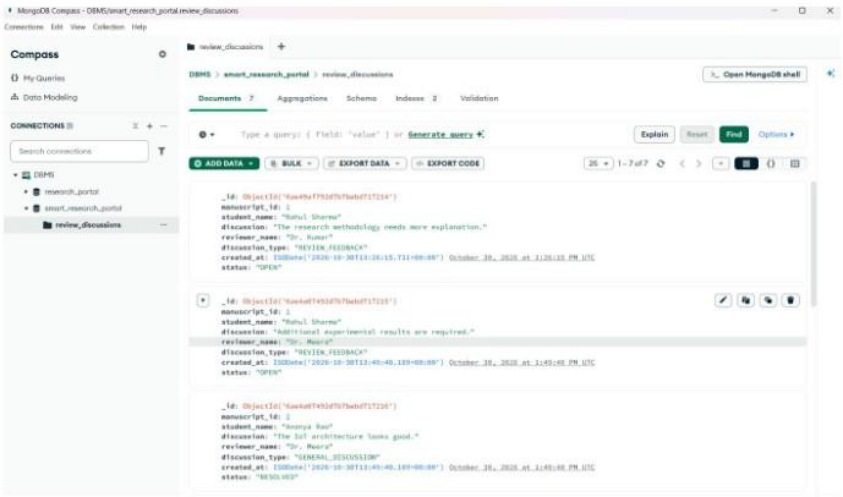
smartportal_db=#
```


Figure 8.2 – SQL JOIN output and trigger/audit evidence.

8. RESULTS – NOSQL EVIDENCE

SMART RESEARCH & PEER REVIEW PORTAL

Screenshot 5 MongoDB



20. EVALUATION RUBRICS – COMPLETION STATUS

The table below is a conservative status summary based on the implementation details available while preparing this report. Recheck each item in the final application and change the status only when verified.

Requirement / feature	Status to verify before submission	Evidence
Minimum five relational tables	Completed – nine physical tables were identified.	\dt output and ER diagram.
Relational database	Implemented – PostgreSQL database smartportal_db.	Database connection and table output.
SQL relationships and constraints	Implemented – primary keys, foreign keys, and unique constraints were identified.	Table definitions and ER diagram.
SQL JOIN operations	Implemented in the student review retrieval query.	API code and query output.
View	Existing view manuscript_review_summary was identified.	\dv and SELECT from view.
Function and trigger	Existing status-change function and trigger were identified.	Trigger/function definition and audit log.
Transactions	Concept and example documented; explicit application workflow not verified.	Show transaction code if implemented.
Student review dashboard	Frontend and API created; verify live retrieval of actual review records.	Running dashboard screenshot.
MongoDB / NoSQL integration	Not confirmed; verify database, collections, and application CRUD.	MongoDB Compass/mongosh and application code.

PES University | DBMS Project Report | Page 16

Figure 8.3 – MongoDB Compass evidence for review_discussions.

The PostgreSQL table-list evidence shows nine physical tables: assignments, audit_logs, departments, manuscript_topics, manuscripts, research_topics, reviewers, reviews, and users. The audit evidence shows the manuscript status-change trigger and two recorded status-change rows in the supplied report.

The shown trigger inserts a NULL user_id. It records the status transition and manuscript identifier but does not identify the authenticated actor.

9. NEXT.JS PROJECT TREE STRUCTURE

The project is implemented using the Next.js App Router. The following tree summarizes the principal directories and files identified during project development. It is a representative structure; compare it with the current repository before submission.

```
Smart_Research_and_Peer_Review_Portal/
|
|-- app/
|   |-- api/
|   |   |-- analytics/
|   |   |-- assignments/
|   |   |-- auth/
|   |   |   |-- login/
|   |   |   |-- logout/
|   |   |   |-- me/
|   |   |   |-- register/
|   |   |-- database-status/
|   |   |-- discussions/
|   |   |-- manuscripts/
|   |   |-- reviewers/
|   |   |-- reviews/
|   |   |-- student/
|   |   |   |-- dashboard/
|   |-- assignments/
|   |-- discussions/
|   |-- login/
|   |-- manuscripts/
|   |-- register/
|   |-- reports/
|   |-- reviewers/
|   |-- reviews/
|   |-- student/
|   |   |-- dashboard/
|   |-- submit/
|
|-- components/
|-- lib/
|   |-- auth.ts
|   |-- db.ts
|   |-- mongodb.ts
|-- public/
|-- .gitignore
|-- middleware.ts
|-- package.json
|-- package-lock.json
|-- README.md
```

9.1 Important Backend Files

File / Module	Responsibility
lib/db.ts	Creates and exports the PostgreSQL Pool using environment variables.
lib/auth.ts	Creates, verifies, reads, and destroys the signed session cookie.
lib/mongodb.ts	Provides the MongoDB connection helper.
app/api/auth/*	Authentication-related API route handlers.
app/api/student/dashboard/route.ts	Student dashboard API endpoint; protected data should be filtered by the authenticated student's ID.
app/student/dashboard/page.tsx	Student-facing review dashboard page.

10. FEW SAMPLE SCREENSHOTS OF APPLICATION CODE AND DEMO

The screenshots below are reproduced from the supplied report. They show the ResearchHub home page, login page, and student review dashboard.

SMART RESEARCH & PEER REVIEW PORTAL

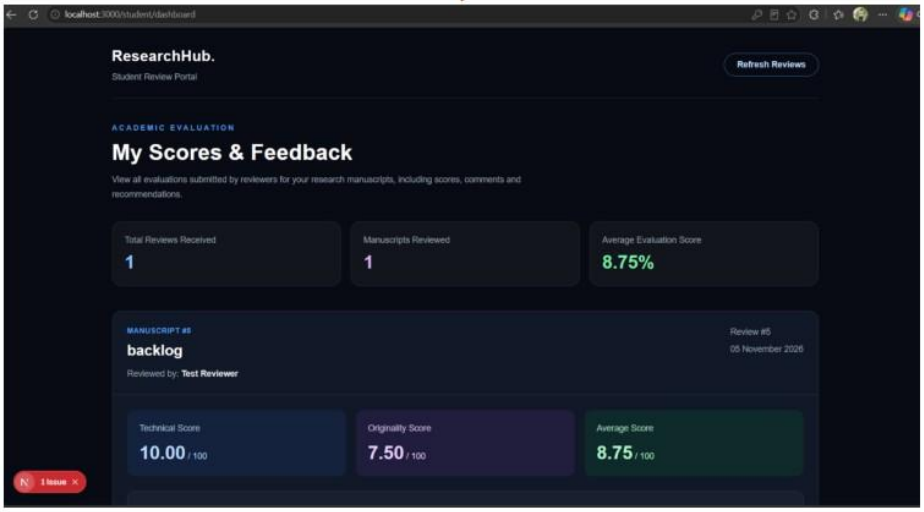
17. STUDENT REVIEW DASHBOARD

The student review page is a single-page view of submitted evaluations. It displays a summary of review count, number of manuscripts reviewed, and average evaluation score, followed by a review card for each submitted review.

Each review card can show manuscript title, reviewer name, technical score, originality score, average score, reviewer comments, recommendation, review date, and assignment identifier.

The page fetches data from /api/student/dashboard. If no review has been submitted, the page displays an empty-state message. Pending assignments are not shown as completed reviews.

The average displayed in the current implementation is calculated from the technical and originality scores. It should not be described as an official final grade unless the project defines that grading policy.



18. TESTING AND VALIDATION

Test	Expected result / evidence
PostgreSQL connection	The application can connect to smartportal_db.
Table verification	The nine expected physical tables appear in \dt.
Student login	A valid student can sign in.
Protected API	Unauthenticated request is rejected.
Role authorization	A non-student role cannot access the student review API.
Student-specific filtering	Only reviews belonging to the session student's manuscripts are returned.
Review retrieval	Submitted review records appear in the API response.
Dashboard display	Scores, comments, recommendation, and date display

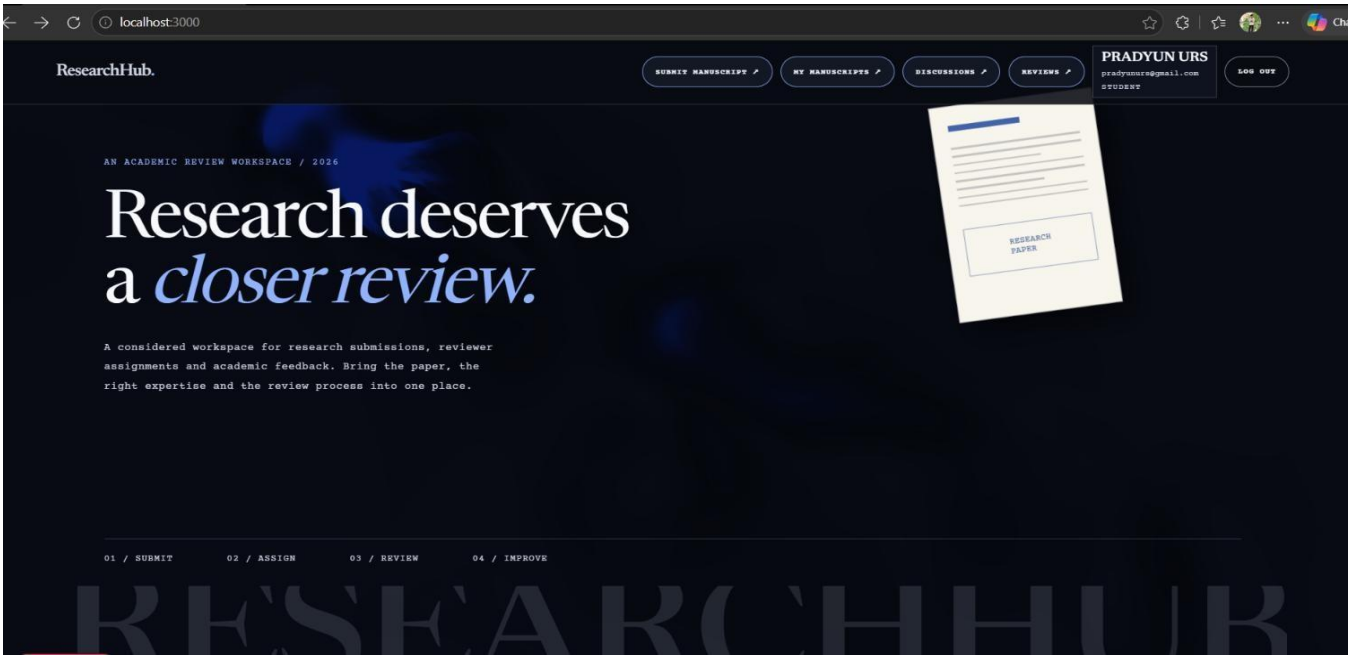
Figure 10.1 – Student review dashboard evidence from the supplied report.

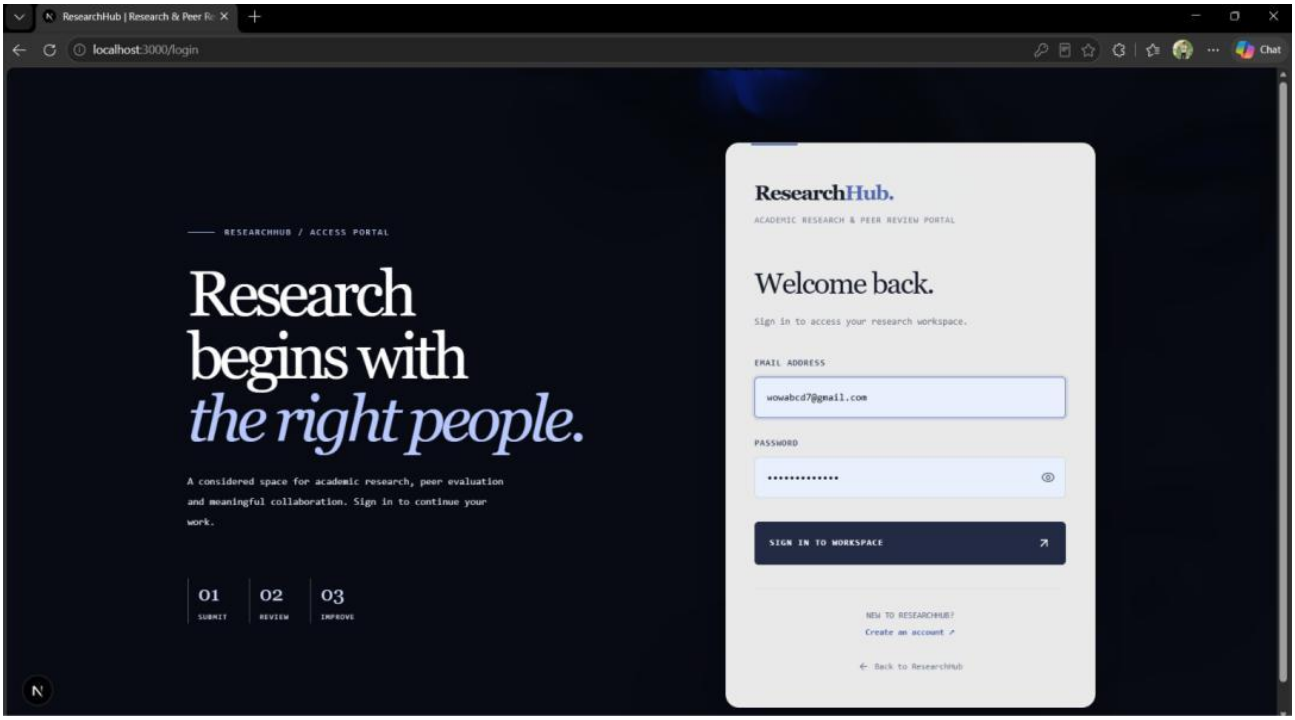
The existing report's page 5 contains the ResearchHub home page and login page screenshots. These are retained

as evidence in the source report; add separate full-resolution application screenshots if the faculty requires them in this section.

10.1 Application and API Behaviour

- The home page provides navigation to manuscript submission, manuscripts, discussions, reviews, and sessionactions as visible in the supplied interface.
- The login page provides email and password fields and a sign-in action.
- The student review dashboard is designed to show review count, manuscript count, average displayed score,and individual feedback cards.
- The student dashboard API should validate the session and STUDENT role, then filter results usingsession.userId.





11. EVALUATION RUBRICS – COMPLETION STATUS

The status below is conservative and reflects the implementation evidence available in the supplied report and project history. Recheck each item in the final running application before submission.

#	Criteria	Required evidence / scope	Status
1	Database Design	ER diagram, relational schema with keys, relationships, and constraints.	Implemented – nine PostgreSQL tables and relationships identified.
2	Database Creation	SQL database with DDL constraints; NoSQL/vector database creation and technology justification.	PostgreSQL implemented. MongoDB collection shown in Compass; application CRUD integration needs verification. pgvector pending.
3	SQL Operations and Queries	DML, nested/correlated/recursive SQL, and JOIN queries.	Basic SELECT, INSERT, UPDATE, JOIN, GROUP BY and aggregate examples documented; nested/correlated/recursive query evidence should be added if required.
4	Advanced Database Concepts	Function/procedure, trigger, CTE, NoSQL/vector feature RUD.	Status-change function and trigger identified. CTE and complete NoSQL/vector CRUD not verified.
5	Web Application and Database Integration	Next.js interface, backend/API and database integration.	Next.js pages/API and PostgreSQL helper present; verify end-to-end operations in live application.
6	Authentication and Authorization	Frontend/backend authentication, minimum two roles, role-based access control.	Authentication/session code and role checks present; verify each role and protected route in live testing.
7	Documentation, Git/GitHub, Demonstration, Testing, and Originality	Documentation, Git/GitHub, originality, individual contribution, testing and demo.	README/report and repository prepared; verify final push, tests, demo, and contribution details.

11.1 Limitations and Future Enhancements

- MongoDB application integration and CRUD operations require final verification.
- pgvector installation was blocked by Windows application-control policy.
- The audit function currently records a NULL user_id for status changes.
- The average of technical and originality scores is a calculated display value, not necessarily an official finalgrade.
- Future work can add actor-aware audit logs, reviewer notifications, advanced reviewer allocation, analytics,downloadable reports, and semantic search after pgvector is operational.

11.2 Conclusion

The Smart Research & Peer Review Portal demonstrates how a relational database can support an academic manuscript-review workflow. PostgreSQL stores structured information about departments, users, research

topics, manuscripts, reviewers, assignments, reviews, and audit logs. The Next.js application communicates with PostgreSQL through the pg connection pool and uses SQL queries to retrieve application data.

The student review feature demonstrates relational joins, foreign-key relationships, parameterized queries, and session-based access control. The database also demonstrates constraints, a view, a stored function, a trigger, and audit logging. The report distinguishes confirmed implementation from planned or pending NoSQL and vector-search capabilities.

REFERENCES

- PostgreSQL Documentation – https://www.postgresql.org/docs/
- Next.js Documentation – https://nextjs.org/docs
- Node.js Documentation – https://nodejs.org/docs/
- node-postgres Documentation – https://node-postgres.com/
- MongoDB Documentation – https://www.mongodb.com/docs/
- JOSE Documentation – https://github.com/panva/jose

— END OF REPORT —